Data Model in Digital Era (DSI463): Week 10

Assignment 4: Evidence-Based Policy from a Document Database (100 points + 20 bonus, 8% of course grade)

- **Database:** `mso` (MongoDB), synthetic MSO-Logbook data on fragile households
- **Hand-out files:** `mso_data.zip` (3 JSON files in `data/`), `load_data.py`, `docker-compose.yml`, `schema_guide.md`, `external_data_sources.md`, `provinces.csv`
- **Team size:** up to 3 students
- **Due:** Thursday 29 October 2026, 23:59 (midnight after the Week 12 lecture)

Background

The Ministry of Social Development and Human Security (พม.) records fragile households in a case-management system called **MSO-Logbook**. Officers and volunteers visit each household and record its housing, income, debt, health, the welfare it already receives, the problems it faces, and the help plans that follow.

The data is stored as **documents**: one household holds families, a family holds members, and each member carries nested arrays of welfare, problems, and plans. This is exactly the kind of data a document database is built for.

You are a data analyst advising a provincial พม. office. **Your job is to find one problem worth solving, prove it with the data, and propose a policy.**

> The data is synthetic but realistic. Patterns in it were designed to resemble real issues. Treat it as if it were real, but never present the numbers as real statistics about Thailand.

Notebook header (copy as your first cell)

```
# Assignment 4: Evidence-Based Policy from a Document Database
**Name(s):** [names]
**Student ID(s):** [IDs]
**Problem title:** [one line]
```

Setup (copy as your second cell)

1. Start MongoDB in one of three ways:
   - **Docker:** run `docker compose up -d` in the hand-out folder (uses `docker-compose.yml`).
   - **Local install:** as in the DSI201 MongoDB Installation slides.
   - **MongoDB Atlas:** a free-tier cluster.
2. Unzip `mso_data.zip` next to `load_data.py`, so the JSON files sit in a `data/` folder.
3. Load the data once:

```
pip install pymongo pandas
python load_data.py                        # or: python load_data.py --uri "<your
Atlas URI>"
```

4. Connect in the notebook:

```
from datetime import datetime
from pymongo import MongoClient
import pandas as pd

db = MongoClient("mongodb://localhost:27017")["mso"]
SNAPSHOT = datetime(2024, 6, 30)
AGE = {"$dateDiff": {"startDate": "$birthdate", "endDate": SNAPSHOT, "unit": "year"}}
ACTIVE = {"status": {"$ne": "removed"}, "vital_status": 1}

def agg(coll, pipeline):
    """Run an aggregation and return a DataFrame."""
    return pd.DataFrame(list(db[coll].aggregate(pipeline)))

for c in ["mso_household", "mso_family", "mso_member"]:
    print(c, db[c].count_documents({}))
```

   Expected output: 2000 / 2217 / 7518.

Read `schema_guide.md` before you start, especially **Section 2: Data-quality notes**.

Task 1: Explore the database (15 points)

Show that you understand the structure before you look for a problem.

1a. Structure (5 pts). Use `find_one()` on each collection and explain in 3–5 sentences how the three collections are linked, and why the `mso_member` documents embed arrays (welfare, problems, plans) instead of storing them in separate collections. Give one advantage and one disadvantage of this design compared with a relational schema.

1b. Overview queries (10 pts). Write **at least four** aggregation pipelines that describe the population. At least one must use `$lookup` and at least one must use `$unwind`. Suggested questions:

- How many active members are in each province, and how many households of each `hh_type`?
- What is the age distribution (use `$bucket` on age)?
- Which ten problem codes are recorded most often across all `prob_*` fields?
- What share of problems in `prob_summary` still have `status: 0` (no plan)?

Show each result as a table and write one sentence on what it tells you.

Task 2: State the problem (20 points, Dimension 1)

Write a **problem statement** of 150–250 words that answers:

1. **Who** is affected? Define the target group precisely using fields in the database (for example, "members aged 60+ in HHT1 households without WFEL9 in `welf_eldery`").
2. **What** is the harm, and why does it matter for a fragile household?
3. **How big** is it? Give a count from the data and a proportion of a meaningful base.
4. **Why now**, and why this group rather than another? Compare it with at least one alternative problem you considered and rejected.

Strong problems usually sit where **two or more domains meet**: for example health and welfare access, debt and education, or housing and disaster.

Task 3: Evidence from the data (30 points, Dimensions 2 and 3)

Build the case with **at least five** aggregation pipelines. Together they must:

- **Size** the target group (count and rate), broken down by at least one location level (province, district).
- **Compare** the target group against a reference group (for example, coverage rate in one province vs the others, or out-of-school rate in skipped-generation vs nuclear families).
- **Profile** the target group across at least three different sections of the member document (economy, health, welfare, problems, case management).

- **Check the response**: use `prob_summary` to show whether the problem is already being handled (problem status, plan types, follow-up results).
- Include at least one `$lookup` across collections and at least one `$unwind` of an embedded array.

For each pipeline, give: the question it answers, the pipeline code, the result (table or chart), and **one sentence of interpretation**.

Field log (required)

End Task 3 with a table listing every field you used as evidence:

#	Collection	Field path	Used in query	Why it matters
1	mso_household	`hh_type.code`	Q2	Defines "living alone"
2	mso_member	`welf_eldery.code`	Q2, Q3	Shows who receives the allowance

A field counts toward Dimension 2 **only if it appears in a pipeline whose result you interpret**. Fields that are only listed, projected, or used to exclude removed records do not count. Nested paths count separately (`debt.type` and `debt.amount` are two fields).

Task 4: Combine external data (20 points, Dimension 4)

Bring in **at least one** external dataset and join it to the database. See `external_data_sources.md` for ideas.

1. Download the data, clean it, and load it into MongoDB as a new collection whose name starts with `ext_` (for example, `ext_province_poverty`). Use `provinces.csv` to match names to `province_code` if your source only has names.
2. Join it with `$lookup` on `province_code` or `amphur_code` , and use it in **at least one** pipeline from Task 3, or a new one.
3. Explain what the external data adds that the database cannot show alone. Examples: the size of the population the logbook does *not* cover, a poverty line to compare per-capita income against, disaster frequency, or the distance to the nearest hospital.
4. Cite the source: publisher, URL, date accessed, and the year the data describes.

Task 5: Policy proposal (15 points, Dimension 1)

Write a **policy brief of at most 2 pages** (export as PDF) containing:

1. **Policy**: what should be done, by whom, for whom. Link it to the solution codelist where possible (for example, "scale up TR47 referrals through อพม. volunteers in districts 5801 and 5803").
2. **Evidence summary**: the three strongest numbers from Task 3 and Task 4.
3. **Targeting**: how a พม. officer would find eligible households with **one MongoDB query**. Include the query.
4. **Cost or effort estimate**: a rough estimate using counts from the data. State your assumptions.
5. **How to measure success**: which fields would change if the policy works, and which query you would run again in six months.
6. **Limitations**: what the data cannot tell you (recording gaps, synthetic data, missing fields).
7. **Looking ahead (optional, not graded, at most 5 sentences).** Weeks 11 and 12 cover full-text search, embeddings and vector databases. Describe one way your policy could use them. Examples: search officers' free-text visit notes for signs of your problem that no code captures; or turn each member's profile into an embedding and find households *similar* to the ones in your target group that were never flagged. Say which data you would need that this database does not yet have (most plan and visit notes here are empty).

Bonus A: Make your queries fast (10 points, optional)

Week 10 showed that a document database is only as fast as its indexes. Apply that to your own work.

A1. One-pipeline dashboard (4 pts). Write **one** aggregation that uses `$facet` to return three views of your target group at once: (a) count by province, (b) count by sex, (c) count by age band (use `$bucket` on age). Put your `$match` stage **before** `$facet`, and explain in one sentence why it belongs there.

A2. Index your targeting query (6 pts). Take the targeting query from Task 5 (item 3).

1. Run it with `explain("executionStats")` **before** adding an index. Record the winning plan stage (`COLLSCAN` / `IXSCAN`), `totalDocsExamined`, `totalKeysExamined`, `nReturned` and `executionTimeMillis`.
2. Create **one compound index** designed with the ESR rule (Equality, Sort, Range). If it includes an array field, say that it is a multikey index.
3. Run `explain("executionStats")` again and show the same numbers side by side in a table.

4. In 3–5 sentences, explain the order of fields in your index, and name any part of the query the index **cannot** help with. For example, `$ne`, a computed age, or stages after `$lookup` cannot use the index.

In `pymongo`, use `db.command("explain", {"find": "mso_member", "filter": {...}}, verbosity="executionStats")`, or `db.mso_member.find(filter).explain()` for a find-style query.

Bonus B: Redesign the MSO-Logbook structure (10 points, optional)

While answering your question, you probably found parts of the database that made it harder than it should be: values you had to compute every time, codes spread across several arrays, information that is copied or missing. Propose how the structure should change so that questions like yours are easier and more reliable to answer.

B1. Diagnose (3 pts). Name **at least two** structural problems you actually ran into. For each, point to the query in your notebook where it caused extra work, a workaround or a risk of error (for example, extra `$lookup` stages, `$concatArrays` over many fields, or `$ifNull` guards).

B2. Redesign (4 pts). Propose the change. Include:

- the new or changed fields, sub-documents or collections, with **one sample document** in JSON;
- for each change, whether you **embed or reference**, and why, based on how the data is read and written (Week 10);
- any index the new structure needs, and one `$jsonSchema` validator rule for a field you think must always be present or valid;
- the cost: what becomes harder (updates, duplication, document size, keeping copies in sync).

B3. Show the benefit (3 pts). Build a small version of your new structure from the existing data (for example, with an aggregation that ends in `$out` or `$merge` into a new collection such as `mso_member_v2`). Then re-run **one** of your Task 3 queries against it and compare it with the original: the number of stages, the result (it must match), and, if you like, `explain()` numbers.

Changes to the content of the data (new survey questions) can be mentioned, but the marks are for **structure**: how the existing information is organised.

Submission

- One submission per team. Put every member's name and student ID in the notebook header and the brief.
- `A4_<teamleaderID>.ipynb` : the notebook with all outputs visible (Tasks 1–4 plus the field log)
- `A4_<teamleaderID>_brief.pdf` : the policy brief (Task 5)
- Any external data files you used, or a script that downloads them

---

Grading rubric

The four dimensions you are graded on are spread across the tasks as shown below.

Dimension	Where it is assessed	Points
1. Importance of the problem/policy	Task 2 (20) + Task 5 (15)	35
2. Number of data fields used	Task 3 field log	15
3. Evidence shown from the data	Task 1 (15) + Task 3 (15)	30
4. External data combined	Task 4	20
Total		**100**
Bonus A: speed (`$facet` + index)	Bonus A	+10
Bonus B: structure redesign	Bonus B	+10

The bonuses can lift a total that lost points elsewhere, but the final score is capped at 100.

Dimension 1: Importance (35)

Level	Task 2 (20)	Task 5 (15)
Excellent	18–20: precise target group defined by fields; harm is serious and well argued; size and rate given; alternative compared	14–15: actionable policy tied to codelist solutions; clear targeting query; realistic cost and success measures
Good	14–17: clear group and harm; size given; comparison is weak	10–13: sensible policy; targeting or measurement is vague
Fair	9–13: group is vague ("poor people"); size missing or unclear	6–9: generic policy not tied to the evidence
Weak	0–8: no clear problem, or a problem the data cannot address	0–5: missing or not a policy

Dimension 2: Fields used (15)

Counted from the field log, using only fields that pass the rule in Task 3.

Distinct fields used as evidence	Points
⩾ 15 fields, from ⩾ 2 collections and ⩾ 3 sections of the member document	15
12–14 fields, or ⩾ 15 from a single section	12
9–11 fields	9
6–8 fields	6
⩽ 5 fields	3

Up to 3 points are deducted if fields are counted that do not contribute to an interpreted result.

Dimension 3: Evidence (30)

Level	Task 1 (15)	Task 3 (15)
Excellent	14–15: correct structural explanation; four correct overview pipelines using `$lookup` and `$unwind`	14–15: all five evidence requirements met; correct handling of removed/deceased/missing values; every result interpreted
Good	10–13: minor errors	10–13: one requirement missing, or one result misread
Fair	6–9: few pipelines or shallow explanation	6–9: counts without comparison; data-quality issues ignored
Weak	0–5	0–5: claims not backed by queries

Dimension 4: External data (20)

Level	Points	Description
Excellent	18–20	Relevant source, properly cited; loaded as an `ext_` collection; joined with `$lookup`; changes or strengthens the conclusion
Good	14–17	Joined and used, but adds only context
Fair	8–13	Used side by side (e.g. quoted in text) but not joined in the database
Weak	0–7	Missing, uncited, or unrelated

Bonus A (10)

Part	Full marks
A1 (4)	One `$facet` pipeline returns all three views correctly; `$match` comes first, with a correct reason (it filters before branching and can use an index)
A2 (6)	Before/after numbers from `executionStats`; an ESR-justified compound index; correct statement of what the index cannot serve. A fair "the index did not help, and here is why" also earns full marks

Bonus B (10)

Part	Full marks
B1 (3)	Two or more real problems, each tied to a specific query in the notebook and the extra work it caused
B2 (4)	Clear sample document; embed/reference choice justified by read and write patterns; index and validator included; honest cost
B3 (3)	Migrated collection built from the real data; the rewritten query returns the same result with a simpler pipeline (or a measured gain)

A redesign that is only described, with no link to the team's own queries, earns at most 4 of the 10 points.

Tips

- Start from **gaps between facts and records**. For example: a person is disabled (`is_disabled: 1`) but has no card (`have_pwd_card: 0`) and no `PBAC2` problem recorded. Who is falling through the net?
- Group members by `hh_id` to get household income and size before computing per-capita income.
- Compare **rates**, not raw counts. Bangkok has more records than Mae Hong Son simply because it has more households in the sample.
- Keep pipelines readable: one stage per line, with a comment on any non-obvious stage.
- The free-text fields in this data (`occup_name` , `disease_name` , `sol_detail` , `act_detail`) are short or empty. That is a real limitation of the logbook, and a good point to make in Task 5 item 7.